Table des matières

Cours 07 — Sous-requêtes, corrélation et ensembles 1
Objectifs 1
Théorie 1
Points clés à retenir 11
Exercices 12

Cours 07 — Sous-requêtes, corrélation et ensembles

Module 7/12 — Formation SQL avancé

Comment lire ce module — C'est le SQL des questions négatives et comparatives, celles que l'agrégation ne sait pas formuler. « Quels produits n'ont jamais été vendus ? », « quels clients ont acheté A mais pas B ? », « quels sont les trois meilleurs de chaque catégorie ? ». Aucune de ces trois questions ne s'écrit avec un `GROUP BY`. La théorie est ici ; les deux laboratoires (`Labo_07_OLAP_SQL.md` et `Labo_07_OLTP_SQL.md`) vous font tout taper.

Objectifs

À la fin de ce module, vous saurez :

- reconnaître les trois formes de sous-requête et savoir où chacune a le droit d'apparaître ;
- distinguer une sous-requête corrélée d'une sous-requête indépendante, et cesser de croire que la première est forcément lente ;
- écrire une semi-jointure et une anti-jointure avec `EXISTS` et `NOT EXISTS`, et expliquer pourquoi `NOT IN` vous trahira un jour ;
- choisir entre `UNION` et `UNION ALL` en sachant ce que le premier coûte ;
- écrire un « top N par groupe » avec `LATERAL`, et le comparer aux deux autres écritures possibles ;
- énoncer ce qui change d'un terrain à l'autre sur ces cinq sujets, et ce qui ne change pas.

Théorie

1. Le problème : ce que le `GROUP BY` ne sait pas dire

© 2026 iAlexMG. Tous droits réservés.

Labo : `Labo_07_OLTP_SQL.md` et `Labo_07_OLAP_SQL.md`, partie 2.

Reprenons la table des ventes. Vous savez compter les commandes par client, sommer le chiffre par catégorie, classer les produits par rang. Toutes ces questions ont un point commun : elles portent sur ce qui **est** dans la table.

Voici maintenant trois questions qui portent sur ce qui **n'y est pas**, ou sur ce qui est ailleurs.

La question	**Pourquoi le `GROUP BY` échoue**
Quels produits n'ont jamais été vendus ?	Un produit jamais vendu n'a aucune ligne de vente. Il ne peut donc apparaître dans aucun groupe d'une agrégation sur les ventes.
Quels clients ont acheté des vélos mais aucun accessoire ?	Il faut comparer deux ensembles de clients, pas agréger un seul.
Quels sont les trois produits les plus chers de chaque sous-catégorie ?	Le `GROUP BY` donne une ligne par groupe. On en veut trois, et pas n'importe lesquelles.

Le point commun des trois réponses : elles ont besoin d'une **seconde requête**, imbriquée dans la première. C'est le sujet du module.

Une mise en garde avant de commencer. La sous-requête est l'outil le plus facile à mal employer du SQL, parce qu'elle permet d'empiler indéfiniment. Le critère qui vous guidera tout au long du module : une sous-requête doit répondre à une question qu'une jointure ne sait pas poser. Si une jointure suffit, écrivez la jointure.

2. Les trois formes de sous-requête

Labo : `Labo_07_OLTP_SQL.md` et `Labo_07_OLAP_SQL.md`, sous-parties 1.1 et 1.3.

Une sous-requête est un `SELECT` entre parenthèses, placé à l'intérieur d'une autre requête. Ce qui change d'une forme à l'autre, c'est **ce qu'elle renvoie**, et cela décide **où elle a le droit d'apparaître**.

Forme	**Ce qu'elle renvoie**	**Où elle vit**	**Ce qui arrive si elle renvoie autre chose**
Scalaire	une seule valeur, une ligne et une colonne	partout où une valeur est attendue : `SELECT`, `WHERE`, `HAVING`	erreur à l'exécution si plusieurs lignes reviennent, `NULL` si aucune
De table	plusieurs lignes et plusieurs colonnes	dans le `FROM`, comme une table	aucune contrainte, c'est une table
De liste	une colonne, plusieurs lignes	avec `IN`, `ANY`, `ALL`, `EXISTS`	erreur si plusieurs colonnes reviennent, sauf avec `EXISTS`

La scalaire. Elle sert à comparer chaque ligne à une valeur globale.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Les produits dont le prix dépasse la moyenne générale.
SELECT p.name, p.listprice
FROM   production.product AS p
WHERE  p.listprice > (SELECT AVG(listprice) FROM production.product);
```

La sous-requête ne dépend de rien d'extérieur : le moteur la calcule **une fois**, garde le résultat et l'utilise pour toutes les lignes. On dit qu'elle est indépendante, ou non corrélée.

Placée dans le `SELECT`, la même forme fabrique une colonne de contexte :

```sql
SELECT p.name,
       p.listprice,
       p.listprice - (SELECT AVG(listprice) FROM production.product) AS ecart_moyenne
FROM   production.product AS p;
```

Notez au passage que cette écriture a un concurrent direct, la fonction de fenêtrage du Cours 04 : `AVG(listprice) OVER ()` produit la même colonne en un seul passage sur la table. Quand les deux sont possibles, la fenêtre est presque toujours préférable, parce qu'elle ne relit pas la table.

La sous-requête de table. Elle sert à préparer un résultat intermédiaire pour la requête principale. Elle exige un alias.

```sql
SELECT s.productid, s.qte_totale
FROM  (SELECT productid, SUM(orderqty) AS qte_totale
       FROM   sales.salesorderdetail
       GROUP  BY productid) AS s
WHERE s.qte_totale > 1000;
```

C'est exactement ce qu'une CTE du Cours 05 fait, en plus lisible. Retenez la règle : dès qu'une sous-requête de table dépasse quelques lignes, ou dès qu'elle sert deux fois, elle doit devenir une CTE. La sous-requête imbriquée se lit de l'intérieur vers l'extérieur, ce qui est contraire au sens de lecture ; la CTE se lit de haut en bas.

La sous-requête de liste. Elle fournit un ensemble de valeurs à comparer, et c'est la section 5 qui la traite en détail.

Piège — Une sous-requête scalaire qui ne renvoie aucune ligne ne provoque pas d'erreur : elle renvoie `NULL`. Une comparaison avec `NULL` n'est ni vraie ni fausse, donc la ligne disparaît silencieusement du résultat. C'est le piège du Cours 01 qui revient, et il est ici plus difficile à voir parce que rien ne le signale.

3. La corrélation

Labo : `Labo_07_OLTP_SQL.md` et `Labo_07_OLAP_SQL.md`, sous-partie 1.2.

Une sous-requête est **corrélée** quand elle lit une colonne de la requête qui l'englobe. Elle ne peut donc plus être calculée une fois pour toutes : sa valeur change d'une ligne à l'autre.

```sql
-- Pour chaque produit, le nombre de fois qu'il a été commandé.
SELECT p.name,
```


© 2026 iAlexMG. Tous droits réservés.

```
        (SELECT COUNT(*)
         FROM   sales.salesorderdetail AS d
         WHERE  d.productid = p.productid) AS nb_lignes   -- p.productid vient du dehors
FROM   production.product AS p;
```

La référence `p.productid` est ce qui fait la corrélation. Conceptuellement, le moteur exécute la sous-requête une fois par ligne du produit : c'est ce qu'on appelle une boucle imbriquée.

Le mythe à démonter, et ce qu'il faut mettre à la place. On lit partout que le corrélé est lent et qu'il faut le réécrire en jointure. C'est trop simple, dans les deux sens. Les moteurs modernes **décorrèlent** une partie de ces requêtes : ils reconnaissent le motif et le transforment en jointure avant même de commencer à lire les données. Mais une partie seulement, et la frontière se lit dans le plan, pas dans l'intuition.

Ce que vous écrivez	Ce que PostgreSQL en fait
`EXISTS` ou `IN` corrélé, dans le `WHERE`	il décorrèle : le plan montre une semi-jointure, `Hash Join` sur un `HashAggregate`, et aucune boucle
`NOT EXISTS` corrélé, dans le `WHERE`	il décorrèle aussi, en anti-jointure
une sous-requête scalaire corrélée, dans le `SELECT`	il ne décorrèle pas : le plan montre un `SubPlan`, relancé pour chaque ligne de la table de gauche

L'écart n'est pas théorique. Sur la base de la formation, le même comptage de lignes de commande par produit coûte, selon l'écriture, 1 603 554 en sous-requête scalaire du `SELECT` contre 3 509 en jointure agrégée : un facteur de plus de quatre cents. Le labo fait lire les deux plans.

Deux conséquences pratiques :

1. N'évitez pas une sous-requête corrélée parce qu'elle est corrélée. Écrivez ce qui exprime le plus clairement la question, puis mesurez.
2. Quand la décorrélation n'a pas lieu, elle se voit dans le plan d'exécution. Le Cours 10 vous apprend à le lire ; d'ici là, retenez seulement que la lenteur se constate, elle ne se devine pas.

Il reste un cas où la corrélation coûte vraiment : quand la sous-requête contient elle-même une agrégation sur une grosse table et que la requête englobante produit beaucoup de lignes. Là, la version en jointure explicite est souvent plus rapide, parce qu'elle calcule l'agrégat une seule fois pour tous les groupes.

4. `EXISTS` et `NOT EXISTS` : semi-jointure et anti-jointure

Labo : `Labo_07_OLTP_SQL.md` et `Labo_07_OLAP_SQL.md`, sous-parties 2.1 et 2.5.

Ce sont les deux outils les plus utiles du module, et ils méritent leurs noms savants.

La semi-jointure répond à « existe-t-il au moins une correspondance ? ». Elle filtre la table du `FROM` d'après une autre table, sans rien ramener de cette autre table : ci-dessous, un produit est gardé s'il apparaît au moins une fois dans les lignes de commande, mais aucune colonne de commande n'arrive dans le résultat.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Les produits qui ont été vendus au moins une fois.
SELECT p.name
FROM   production.product AS p
WHERE  EXISTS (SELECT 1
               FROM   sales.salesorderdetail AS d
               WHERE  d.productid = p.productid);
```

L'anti-jointure répond à la question inverse, et c'est elle qui résout la première énigme de la section 1.

```sql
-- Les produits jamais vendus.
SELECT p.name
FROM   production.product AS p
WHERE  NOT EXISTS (SELECT 1
                   FROM   sales.salesorderdetail AS d
                   WHERE  d.productid = p.productid);
```

Trois remarques sur la forme.

Le `SELECT 1` n'est pas une astuce, c'est une déclaration d'intention : `EXISTS` ne regarde jamais les colonnes renvoyées, seulement s'il y a au moins une ligne. Écrire `SELECT *` marcherait identiquement ; `SELECT 1` dit au lecteur qu'on ne s'intéresse pas au contenu.

`EXISTS` s'arrête à la première ligne trouvée. Il n'a pas besoin de compter, contrairement à un `COUNT(*) > 0` qui, lui, parcourt tout.

`EXISTS` accepte plusieurs colonnes dans son `SELECT`, puisqu'il les ignore. C'est la seule forme de sous-requête de liste qui l'autorise.

Pourquoi `NOT EXISTS` bat `NOT IN`. C'est le point le plus important du module, et il découle directement du piège `NULL` du Cours 01.

```sql
-- Écriture fautive, si la colonne peut contenir NULL.
SELECT p.name
FROM   production.product AS p
WHERE  p.productid NOT IN (SELECT d.productid FROM sales.salesorderdetail AS d);
```

Si la liste renvoyée contient ne serait-ce qu'un seul `NULL`, cette requête ne renvoie **jamais rien**, quelle que soit la donnée. La raison est logique et implacable. `x NOT IN (a, b, NULL)` se développe en `x <> a AND x <> b AND x <> NULL`. Le dernier terme vaut `UNKNOWN`, jamais `TRUE`. Un `AND` avec `UNKNOWN` ne peut plus valoir `TRUE`, donc aucune ligne ne passe le filtre.

Le plus mauvais dans cette affaire : la requête ne produit pas d'erreur. Elle renvoie zéro ligne, et zéro ligne est une réponse plausible à « quels produits n'ont jamais été vendus ». Vous pouvez donc livrer un résultat faux sans jamais le savoir.

Écriture	Colonne sans `NULL`	Colonne avec `NULL`
`NOT EXISTS`	correcte	correcte

© 2026 iAlexMG. Tous droits réservés.

Écriture	Colonne sans `NULL`	Colonne avec `NULL`
`NOT IN`	correcte	**renvoie zéro ligne**, en silence
`LEFT JOIN … WHERE clé IS NULL`	correcte	correcte

La règle à retenir tient en une phrase : pour une anti-jointure, on écrit `NOT EXISTS`, toujours. `NOT IN` ne se justifie que sur une liste littérale écrite à la main, où l'on voit qu'il n'y a pas de `NULL`.

La troisième écriture, le `LEFT JOIN` suivi d'un test `IS NULL` sur la clé de la table de droite, est correcte elle aussi et vous la rencontrerez souvent. Elle est plus verbeuse et son intention est moins lisible : il faut avoir compris que le `IS NULL` teste l'absence de correspondance, pas une vraie valeur nulle.

Portabilité — DuckDB écrit aussi la semi-jointure et l'anti-jointure en toutes lettres. `FROM DimProduct AS p ANTI JOIN FactInternetSales AS f ON f.ProductKey = p.ProductKey` rend les 448 lignes de produits jamais vendus en ligne, `SEMI JOIN` les 158 autres, sans doublon et sans colonne de la table de droite. PostgreSQL ne connaît pas ces mots, et c'est là que la faute coûte cher. Avec un alias, `FROM production.product AS p ANTI JOIN …` lève une erreur de syntaxe, et tout va bien. Sans alias, `FROM production.product ANTI JOIN sales.salesorderdetail USING (productid)` passe sans un mot : PostgreSQL lit `ANTI` comme l'alias de `production.product`, fait une jointure interne et rend les 121 317 lignes de commande, exactement celles qu'on voulait écarter. Mesuré sur nos deux terrains : la même ligne rend l'anti-jointure sur l'un et son contraire sur l'autre. `NOT EXISTS` est la forme qui voyage.

5. `IN`, `ANY`, `ALL`

Labo : `Labo_07_OLTP_SQL.md` et `Labo_07_OLAP_SQL.md`, sous-partie 2.2.

Ces trois mots-clés comparent une valeur à un ensemble. Ils sont plus liés qu'il n'y paraît.

Écriture	Équivalent	Se lit
`x IN (…)`	`x = ANY (…)`	égal à au moins un
`x NOT IN (…)`	`x <> ALL (…)`	différent de tous
`x > ANY (…)`		plus grand qu'au moins un, donc plus grand que le minimum
`x > ALL (…)`		plus grand que tous, donc plus grand que le maximum

`ANY` et `ALL` acceptent n'importe quel opérateur de comparaison, ce qui les rend plus expressifs que `IN`. En pratique on les voit peu, parce que la version avec `MIN` ou `MAX` se lit mieux : `x > (SELECT MAX(…) …)` est plus directe à lire que `x > ALL (…)`.

Plus directe, mais pas équivalente, et c'est un piège de plus à ajouter à votre collection. Les deux écritures divergent précisément dans les deux cas limites que ce module enseigne, et elles divergent en sens inverse l'une de l'autre.

© 2026 iAlexMG. Tous droits réservés.

Cas	`x > ALL (…)`	`x > (SELECT MAX(…))`
L'ensemble est vide	**vrai** : rien à contredire	**inconnu** : `MAX` d'un ensemble vide vaut `NULL`
L'ensemble contient un `NULL`	**inconnu** : une comparaison avec `NULL` contamine le tout	**vrai** : `MAX` ignore les `NULL`

Mesuré sur nos deux terrains, qui répondent identiquement. Retenez la logique plutôt que le tableau : `ALL` regarde chaque élément, donc un `NULL` le rend inconnu et une liste vide ne peut rien lui opposer ; `MAX` est un agrégat, donc il ignore les `NULL` et rend `NULL` quand il n'a rien à agréger. Ce ne sont pas deux façons d'écrire la même chose, ce sont deux questions différentes qui coïncident tant que les données sont propres.

Le choix pratique en découle. Si l'ensemble peut être vide et que vous voulez alors que la condition passe, écrivez `ALL`. Si l'ensemble peut contenir des `NULL` que vous voulez ignorer, écrivez `MAX`. Et dans le doute, faites ce que fait le reste du module : posez la question à voix haute et regardez laquelle des deux réponses vous vouliez.

Piège — `ALL` sur un ensemble vide vaut `TRUE`, et `ANY` sur un ensemble vide vaut `FALSE`. Ce n'est pas une bizarrerie de moteur, c'est la logique : « plus grand que tous les éléments » est vrai quand il n'y a aucun élément à contredire. Une requête bâtie sur `ALL` peut donc laisser passer toutes les lignes le jour où sa sous-requête ne renvoie plus rien.

6. Les opérateurs d'ensembles

Labo : `Labo_07_OLTP_SQL.md` et `Labo_07_OLAP_SQL.md`, sous-parties 2.3 et 2.6.

Là où la jointure combine des colonnes, l'opérateur d'ensemble empile des **lignes**. Il travaille sur deux résultats entiers.

Opérateur	Ce qu'il renvoie	Dédoublonne
`UNION ALL`	toutes les lignes des deux côtés	non
`UNION`	les lignes des deux côtés	oui
`INTERSECT`	les lignes présentes des deux côtés	oui
`EXCEPT`	les lignes du premier absentes du second	oui
`EXCEPT ALL`	les lignes du premier en trop par rapport au second, répétitions comptées	non

Trois règles de compatibilité, valables pour les quatre : même nombre de colonnes, types compatibles colonne par colonne, et l'ordre des colonnes fait foi. Les **noms**, eux, ne comptent pas : ce sont ceux du premier `SELECT` qui nomment le résultat. C'est une source d'erreur discrète, puisque deux colonnes mal alignées mais de même type s'empileront sans un mot.

© 2026 iAlexMG. Tous droits réservés.

Portabilité — L'entrepôt offre une parade : `UNION BY NAME` et `UNION ALL BY NAME` alignent les colonnes par leur nom plutôt que par leur place. Mesuré sur DuckDB, `SELECT 1 AS a, 2 AS b UNION ALL BY NAME SELECT 2 AS b, 1 AS a` rend deux fois la ligne 1, 2 ; PostgreSQL refuse la forme, par une erreur de syntaxe. Snowflake l'a aussi, d'après sa documentation. Là où elle manque, la seule parade reste de relire l'ordre des colonnes.

Le coût caché du dédoublonnage. `UNION`, `INTERSECT` et `EXCEPT` éliminent les doublons, ce qui oblige le moteur à trier ou à hacher l'ensemble du résultat. Sur des millions de lignes, c'est l'opération la plus chère de la requête. `UNION ALL` ne fait qu'empiler et ne coûte presque rien.

D'où la règle : **écrivez `UNION ALL` par défaut**, et ne passez à `UNION` que si vous savez qu'il y a des doublons et que vous voulez vraiment les supprimer. L'inverse, l'habitude d'écrire `UNION` partout par précaution, est un des gaspillages les plus répandus en SQL.

C'est `EXCEPT` qui résout la deuxième énigme de la section 1 :

```sql
-- Les clients qui ont acheté des vélos, mais aucun accessoire.
SELECT customerid FROM ventes_par_categorie WHERE categorie = 'Bikes'
EXCEPT
SELECT customerid FROM ventes_par_categorie WHERE categorie = 'Accessories';
```

Notez que `EXCEPT` compare des lignes entières, pas une clé. Si vous ajoutez une colonne à l'un des deux `SELECT`, la comparaison change de sens sans prévenir.

Comparer dans les deux sens. `A EXCEPT B` rend ce que `A` a de plus que `B`, jamais ce qui lui manque. Une seule direction répond donc à une moitié de la question, et la moitié qu'on oublie est souvent celle qui compte. Les clients de 2012 et de 2013 sur l'entrepôt :

```sql
WITH clients AS (
    SELECT DISTINCT CustomerKey, YEAR(OrderDate) AS annee
    FROM   FactInternetSales
)
SELECT (SELECT COUNT(*) FROM (SELECT CustomerKey FROM clients WHERE annee = 2012
                              EXCEPT
                              SELECT CustomerKey FROM clients WHERE annee = 2013)) AS perdus,
       (SELECT COUNT(*) FROM (SELECT CustomerKey FROM clients WHERE annee = 2013
                              EXCEPT
                              SELECT CustomerKey FROM clients WHERE annee = 2012)) AS gagnes;
```

287 clients de 2012 ne sont pas revenus en 2013, et 14 461 clients de 2013 n'avaient rien acheté en 2012. Une seule des deux requêtes aurait raconté une histoire fausse : la première, une clientèle qui s'effrite ; la seconde, une croissance sans pertes. C'est la même forme qui rapproche une copie de sa source, un extrait de la base, une table rechargée de celle d'hier : zéro dans les deux sens, et seulement là, les deux ensembles sont égaux.

`EXCEPT` dédoublonne, et c'est un défaut dès qu'on rapproche des lignes plutôt que des valeurs. Une copie qui a reçu deux fois une partie de sa source contient les mêmes valeurs, en plus grand nombre : `EXCEPT` répond zéro dans les deux sens, et `EXCEPT ALL`, qui soustrait occurrence par occurrence, rend les lignes en trop. Le labo le mesure sur une journée de ventes dont un tiers des lignes est chargé deux fois. Enfin, les opérateurs d'ensembles comparent les `NULL` comme des égaux, au contraire de `=` : deux lignes `(1, NULL)` se reconnaissent et s'annulent dans un `EXCEPT`, là où une anti-jointure écrite `WHERE y.b = x.b` les croirait différentes. Pour un rapprochement, c'est le comportement qu'on veut.

© 2026 iAlexMG. Tous droits réservés.

Le comparatif avec les autres outils. Trois façons d'exprimer une anti-jointure coexistent maintenant dans votre boîte : `NOT EXISTS`, `EXCEPT`, et le `LEFT JOIN` avec test `IS NULL`. Elles ne sont pas interchangeables.

Outil	Compare	Bon quand
`NOT EXISTS`	une condition de correspondance, souple	on filtre une table par l'absence dans une autre, et on veut garder toutes les colonnes de la table filtrée
`EXCEPT`	des lignes entières, colonne à colonne	les deux côtés ont la même forme et on veut la différence d'ensembles
`LEFT JOIN` avec `IS NULL`	une clé de jointure	on a de toute façon besoin des colonnes de la table de droite ailleurs dans la requête

7. `LATERAL` : la jointure qui regarde à gauche

Labo : `Labo_07_OLTP_SQL.md` et `Labo_07_OLAP_SQL.md`, sous-partie 2.4.

Voici la dernière énigme de la section 1 : les trois produits les plus chers de **chaque** sous-catégorie.

Une sous-requête ordinaire placée dans le `FROM` ne peut pas voir les autres tables du `FROM`. Elle est calculée dans le vide, avant elles. C'est précisément ce que `LATERAL` change : le mot-clé autorise la sous-requête à référencer ce qui la précède dans la clause `FROM`, autrement dit ce qui est écrit à sa gauche. Ci-dessous, la table de gauche est `productsubcategory` ; la sous-requête, écrite à sa droite, lit sa colonne `sc.productsubcategoryid`.

Rappel — « Gauche » et « droite » décrivent l'ordre d'écriture de la requête, pas la position d'une ligne dans une table (Cours 01, section 7). La table de gauche est celle écrite avant le `JOIN`, la table de droite celle écrite après.

```sql
-- Les 3 produits les plus chers de chaque sous-catégorie.
SELECT sc.name AS sous_categorie, top.name, top.listprice
FROM   production.productsubcategory AS sc
LEFT JOIN LATERAL (
          SELECT p.name, p.listprice
          FROM   production.product AS p
          WHERE  p.productsubcategoryid = sc.productsubcategoryid   -- visible grâce à LATERAL
          ORDER  BY p.listprice DESC
          LIMIT  3
        ) AS top ON true;
```

Deux détails de syntaxe méritent qu'on s'y arrête. Le `ON true` surprend : `LATERAL` est déjà une jointure, mais la condition de rapprochement vit à l'intérieur de la sous-requête, dans son `WHERE`. Il ne reste donc rien à mettre dans le `ON`, et la syntaxe exige quand même quelque chose. Et le `LEFT` sert à garder les sous-catégories vides : avec un `JOIN` simple, une sous-catégorie sans produit disparaîtrait du résultat.

© 2026 iAlexMG. Tous droits réservés.

Trois écritures pour le même besoin. Le « top N par groupe » est un classique d'entretien, et vous savez maintenant l'écrire de trois façons.

Écriture	Comment	Sa force	Sa faiblesse
Fonction de fenêtrage (Cours 04)	`ROW_NUMBER() OVER (PARTITION BY … ORDER BY …)` puis filtrer le rang	un seul passage sur la table, la plus rapide en général	il faut une couche de plus (CTE ou sous-requête) pour filtrer le rang, sauf avec `QUALIFY`
`QUALIFY`	la même chose, avec le filtre du rang directement dans la requête	la plus courte à écrire	n'existe que sur l'entrepôt, pas sur PostgreSQL
`LATERAL`	une sous-requête ordonnée et limitée, exécutée pour chaque ligne de la table de gauche	exprime littéralement « pour chaque groupe, les N premiers », et accepte n'importe quelle logique	une exécution par ligne de la table de gauche, coûteux si cette table est grosse

En pratique, la fenêtre gagne quand il s'agit vraiment d'un classement. `LATERAL` gagne quand la sous-requête fait autre chose qu'un classement, par exemple aller chercher la dernière valeur connue avant une date. C'est exactement le geste de l'as-of join, et c'est pourquoi ce module est le prérequis du Cours 09.

8. Sur les deux terrains

Labo : `Labo_07_OLTP_SQL.md` et `Labo_07_OLAP_SQL.md`, partie 3.

Sujet	PostgreSQL (voie OLTP)	DuckDB (voie OLAP)
`LATERAL`	le mot-clé est obligatoire : sans lui, « référence invalide d'une entrée de la clause FROM », avec l'astuce qui nomme le remède	le mot-clé est facultatif, une sous-requête du `FROM` voit la précédente sans qu'on l'annonce
`EXCEPT`	`EXCEPT` et `EXCEPT ALL`	les deux aussi
`INTERSECT`	`INTERSECT` et `INTERSECT ALL`	les deux aussi
La semi- et l'anti-jointure en toutes lettres	absentes ; `FROM t ANTI JOIN …` sans alias passe sans erreur, `ANTI` lu comme un alias	`SEMI JOIN` et `ANTI JOIN`
Le piège `NOT IN`	se manifeste pour de vrai, les colonnes nullables sont partout dans un schéma normalisé	identique, et l'entrepôt a ses propres nullables
Top N par groupe	fenêtre plus CTE, ou `LATERAL`	fenêtre plus `QUALIFY`, la forme la plus courte des deux terrains
`LIMIT`	`LIMIT n`, ou `FETCH FIRST n ROWS ONLY`	les deux aussi

© 2026 iAlexMG. Tous droits réservés.

Trois remarques sur ce tableau. La première ligne est un écart de syntaxe qui ne coûte rien : écrire `LATERAL` partout fonctionne des deux côtés, et PostgreSQL vous rappelle de l'écrire quand vous l'oubliez. Les versions `ALL` des opérateurs d'ensemble, qu'on croit souvent réservées à PostgreSQL, passent sur les deux terrains ; `MINUS`, le synonyme d'`EXCEPT` qu'on rencontre sur Oracle et sur Snowflake d'après leur documentation, n'existe en revanche sur aucun des deux et rend une erreur de syntaxe. La dernière ligne, enfin, est l'écart le plus dangereux du cours : une écriture de DuckDB que PostgreSQL ne refuse pas toujours, et dont il renverse le sens sans prévenir.

Rappel — `FETCH FIRST n ROWS ONLY`, la forme normalisée de `LIMIT`, est présentée au Cours 02, section 1.

Ce qui ne change pas, et c'est l'essentiel : la logique des trois valeurs, donc le piège de `NOT IN`, est identique des deux côtés. C'est du standard SQL, pas une particularité de moteur.

Points clés à retenir

- Une sous-requête doit répondre à une question qu'une jointure ne sait pas poser. Si une jointure suffit, écrivez la jointure.
- Trois formes : scalaire (une valeur), de table (dans le `FROM`, avec alias obligatoire), de liste (avec `IN`, `ANY`, `ALL`, `EXISTS`). Ce qu'elle renvoie décide où elle peut vivre.
- Une sous-requête scalaire sans résultat renvoie `NULL`, pas une erreur, et la ligne disparaît en silence.
- Corrélée veut dire qu'elle lit une colonne du dehors. Ce n'est pas synonyme de lent : les moteurs décorrèlent la plupart des cas. La lenteur se mesure, elle ne se devine pas.
- `EXISTS` est la semi-jointure, `NOT EXISTS` l'anti-jointure. `EXISTS` s'arrête à la première ligne trouvée et ignore les colonnes, d'où le `SELECT 1`.
- `NOT IN` sur une colonne nullable renvoie zéro ligne, sans erreur et sans avertissement. Pour une anti-jointure, on écrit `NOT EXISTS`, toujours.
- `ALL` sur un ensemble vide vaut vrai, `ANY` sur un ensemble vide vaut faux.
- `UNION ALL` est le défaut. `UNION`, `INTERSECT` et `EXCEPT` dédoublonnent, donc trient, donc coûtent. Les noms de colonnes ne comptent pas, seul l'ordre fait foi.
- `LATERAL` autorise une sous-requête du `FROM` à voir ce qui la précède. C'est l'écriture littérale du « pour chaque ligne de la table de gauche, va chercher… », et le prérequis de l'as-of join du Cours 09.
- Le top N par groupe s'écrit de trois façons : fenêtre, `QUALIFY` sur l'entrepôt, ou `LATERAL`. La fenêtre gagne pour un classement, `LATERAL` gagne quand la sous-requête fait autre chose.

© 2026 iAlexMG. Tous droits réservés.

Exercices

Les exercices de ce module et leur corrigé vivent dans `Exercices_corriges_07_SQL.md`. Faites-les après les laboratoires : plusieurs énoncés supposent que les requêtes ont été tapées.

© 2026 iAlexMG. Tous droits réservés.